

K-Means Clustering

Line-by-Line Explanation

Based on the notebook: [kmeans-clustering.ipynb](#)

This PDF explains every code cell in beginner-friendly language, including the purpose of each line, the machine learning concept behind it, and important corrections/improvements.

Topic	Unsupervised Machine Learning - K-Means Clustering
Dataset 1	Student clustering using CGPA and IQ
Dataset 2	Synthetic 3D clustering using make_blobs
Prepared by	ChatGPT for Syed Uzair Hussain

Notebook Overview

This notebook demonstrates K-Means clustering in two stages. First, it clusters students using two features: CGPA and IQ. Second, it generates artificial 3D data and applies K-Means to visualize clusters in three dimensions.

- Part 1: Load student data, visualize it, find the best number of clusters using the Elbow Method, and create 4 clusters.
- Part 2: Generate synthetic 3D data, visualize it using Plotly, again use the Elbow Method, and create final 3D clusters.

Important Concepts Before Starting

K-Means Clustering: K-Means is an unsupervised machine learning algorithm. It groups similar data points together without using pre-defined labels.

Cluster: A cluster is a group of similar data points. In this notebook, students with similar CGPA and IQ values may form one cluster.

Centroid: A centroid is the center point of a cluster. K-Means repeatedly updates centroids until the clusters become stable.

WCSS / Inertia: WCSS means Within-Cluster Sum of Squares. In Scikit-learn, it is stored in `km.inertia_`. Lower WCSS means points are closer to their cluster centers.

Elbow Method: The Elbow Method helps choose the best value of K by plotting number of clusters against WCSS. The bending point of the graph is considered a good value of K.

Cell 0: Importing Required Libraries

```
import numpy as np
import pandas as pd
```

Line-by-line Explanation

import numpy as np

This imports the NumPy library. NumPy is used for numerical calculations, arrays, and matrix-style data handling. The short name np is used so we can write np.array() instead of numpy.array().

import pandas as pd

This imports the Pandas library. Pandas is used for loading, cleaning, and analyzing tabular data such as CSV files. The short name pd is used so we can write pd.read_csv() instead of pandas.read_csv().

Cell 1: Loading the Student Dataset

```
df = pd.read_csv('data/student_clustering.csv')
print("The shape of data is",df.shape)
df.head()
```

Line-by-line Explanation

df = pd.read_csv('data/student_clustering.csv')

This reads the CSV file named student_clustering.csv from the data folder. The data is stored in a Pandas DataFrame named df. A DataFrame is like an Excel sheet with rows and columns.

print("The shape of data is",df.shape)

This prints the size of the dataset. df.shape returns two values: number of rows and number of columns. For example, (200, 2) means 200 records and 2 columns.

df.head()

This displays the first 5 rows of the dataset. It is used to quickly check the column names and sample values, such as cgpa and iq.

Cell 2: Creating a Scatter Plot of CGPA and IQ

```
import matplotlib.pyplot as plt
plt.scatter(df['cgpa'],df['iq'])
plt.show()
```

Line-by-line Explanation

import matplotlib.pyplot as plt

This imports Matplotlib Pyplot, which is used for creating graphs. The alias plt makes plotting commands shorter.

plt.scatter(df['cgpa'],df['iq'])

This creates a scatter plot. The CGPA column is placed on the x-axis and the IQ column is placed on the y-axis. Each dot represents one student.

plt.show()

This displays the graph on the screen. Without this line, some environments may not show the plot properly.

Cell 3: Importing the KMeans Algorithm

```
from sklearn.cluster import KMeans
```

Line-by-line Explanation

```
from sklearn.cluster import KMeans
```

This imports the KMeans clustering algorithm from Scikit-learn. K-Means is used to divide data into groups based on similarity. It is unsupervised because we do not provide labels during training.

Cell 4: Calculating WCSS for Different Values of K

```
wcss = []

for i in range(1,11):
    km = KMeans(n_clusters=i)
    km.fit_predict(df)
    wcss.append(km.inertia_)
```

Line-by-line Explanation

```
wcss = []
```

This creates an empty list named wcss. The list will store WCSS values for different numbers of clusters.

```
for i in range(1,11):
```

This loop tests K values from 1 to 10. range(1,11) produces 1, 2, 3, ..., 10.

```
km = KMeans(n_clusters=i)
```

This creates a K-Means model with i clusters. When i is 1, the model makes 1 cluster. When i is 4, the model makes 4 clusters.

```
km.fit_predict(df)
```

This trains the K-Means model on the DataFrame df and assigns a cluster number to each student. The predictions are not stored here because the purpose is only to calculate WCSS.

```
wcss.append(km.inertia_)
```

This stores the inertia value of the model in the wcss list. Inertia is the WCSS value calculated by Scikit-learn.

Cell 5: Displaying the WCSS List

```
wcss
```

Line-by-line Explanation

```
wcss
```

This displays all WCSS values calculated for K = 1 to K = 10. Each value corresponds to one number of clusters.

Cell 6: Plotting the Elbow Graph

```
plt.plot(range(1,11),wcss)
```

Line-by-line Explanation

`plt.plot(range(1,11),wcss)`

This plots the Elbow Graph. The x-axis contains the number of clusters from 1 to 10. The y-axis contains WCSS values. The best K is usually where the graph bends like an elbow.

Cell 7: Applying Final K-Means with 4 Clusters

```
X = df.iloc[:, :].values
km = KMeans(n_clusters=4)
y_means = km.fit_predict(X)
```

Line-by-line Explanation

`X = df.iloc[:, :].values`

This selects all rows and all columns from df and converts the DataFrame into a NumPy array. Scikit-learn models commonly work with NumPy arrays.

`km = KMeans(n_clusters=4)`

This creates a K-Means model with 4 clusters. The value 4 is chosen because the Elbow Method suggests that 4 is a suitable number of clusters.

`y_means = km.fit_predict(X)`

This trains the model on X and assigns each data point to one of the 4 clusters. The cluster labels are stored in y_means.

Cell 8: Displaying Cluster Labels

```
y_means
```

Line-by-line Explanation

`y_means`

This displays the cluster label assigned to each student. The labels are usually 0, 1, 2, and 3. These numbers are cluster IDs, not marks or grades.

Cell 9: Selecting Students in Cluster 0

```
X[y_means == 0]
```

Line-by-line Explanation

`X[y_means == 0]`

This selects all rows from X where the corresponding value in y_means is 0. In simple words, it displays all students belonging to cluster 0.

Cell 10: Selecting Students in Cluster 1

```
X[y_means == 1]
```

Line-by-line Explanation

`X[y_means == 1]`

This selects all students whose assigned cluster label is 1.

Cell 11: Selecting Students in Cluster 2

```
X[y_means == 2]
```

Line-by-line Explanation

```
X[y_means == 2]
```

This selects all students whose assigned cluster label is 2.

Cell 12: Selecting Students in Cluster 3

```
X[y_means == 3]
```

Line-by-line Explanation

```
X[y_means == 3]
```

This selects all students whose assigned cluster label is 3.

Cell 13: Selecting CGPA Values of Cluster 3

```
X[y_means == 3,0]
```

Line-by-line Explanation

```
X[y_means == 3,0]
```

This selects the first column values for students in cluster 3. Since the first column is likely CGPA, this returns the CGPA values of cluster 3 students.

Cell 14: Selecting IQ Values of Cluster 3

```
X[y_means == 3,1]
```

Line-by-line Explanation

```
X[y_means == 3,1]
```

This selects the second column values for students in cluster 3. Since the second column is likely IQ, this returns the IQ values of cluster 3 students.

Cell 15: Visualizing the Final 2D Clusters

```
plt.scatter(X[y_means == 0,0],X[y_means == 0,1],color='blue')  
plt.scatter(X[y_means == 1,0],X[y_means == 1,1],color='red')  
plt.scatter(X[y_means == 2,0],X[y_means == 2,1],color='green')
```

```
plt.scatter(X[y_means == 3,0],X[y_means == 3,1],color='yellow')  
plt.show()
```

Line-by-line Explanation

```
plt.scatter(X[y_means == 0,0],X[y_means == 0,1],color='blue')
```

This plots cluster 0. The first selected column is used as the x-axis and the second selected column is used as the y-axis. The points are colored blue.

```
plt.scatter(X[y_means == 1,0],X[y_means == 1,1],color='red')
```

This plots cluster 1 in red.

```
plt.scatter(X[y_means == 2,0],X[y_means == 2,1],color='green')
```

This plots cluster 2 in green.

```
plt.scatter(X[y_means == 3,0],X[y_means == 3,1],color='yellow')
```

This plots cluster 3 in yellow.

```
plt.show()
```

This displays the final graph where each color represents one student group.

Cell 16: Markdown Heading: K-Means on 3D Data

```
# K-Means on 3-D Data
```

Line-by-line Explanation

```
# K-Means on 3-D Data
```

This is a Markdown heading. It introduces the second part of the notebook, where K-Means is applied to three-dimensional data.

Cell 17: Generating Synthetic 3D Data

```
from sklearn.datasets import make_blobs

centroids = [(-5, -5, 5), (5, 5, -5), (3.5, -2.5, 4), (-2.5, 2.5, -4)]
cluster_std = [1, 1, 1, 1]

X, y =
make_blobs(n_samples=200, cluster_std=cluster_std, centers=centroids, n_features=3, random_state=
1)
```

Line-by-line Explanation

```
from sklearn.datasets import make_blobs
```

This imports make_blobs from Scikit-learn. make_blobs is used to generate artificial clustered data for practice and visualization.

```
centroids = [(-5, -5, 5), (5, 5, -5), (3.5, -2.5, 4), (-2.5, 2.5, -4)]
```

This defines 4 center points for 4 clusters. Each center has 3 values because the data is three-dimensional: x, y, and z.

```
cluster_std = [1, 1, 1, 1]
```

This defines the spread of each cluster. A smaller value makes the cluster tighter, while a larger value makes it more scattered. Here all clusters have equal spread of 1.

```
X, y =
make_blobs(n_samples=200, cluster_std=cluster_std, centers=centroids, n_features=3, random_state=1)
```

This generates 200 data points using the defined centers. n_features=3 means each point has three coordinates. random_state=1 makes the generated data reproducible. X stores the data points and y stores the original generated labels.

Cell 18: Displaying the 3D Data Array

```
X
```

Line-by-line Explanation

```
X
```

This displays the generated 3D data. Each row has three values representing x, y, and z coordinates.

Cell 19: Creating an Interactive 3D Scatter Plot

```
import plotly.express as px
fig = px.scatter_3d(x=X[:,0], y=X[:,1], z=X[:,2])
fig.show()
```

Line-by-line Explanation

`import plotly.express as px`

This imports Plotly Express, a library for interactive visualizations. The alias `px` is used for shorter commands.

`fig = px.scatter_3d(x=X[:,0], y=X[:,1], z=X[:,2])`

This creates a 3D scatter plot. `X[:,0]` is used for the x-axis, `X[:,1]` for the y-axis, and `X[:,2]` for the z-axis.

`fig.show()`

This displays the interactive 3D plot. In a notebook, you can rotate and zoom the graph.

Cell 20: Calculating WCSS for 3D Data

```
wcss = []
for i in range(1,11):
    km = KMeans(n_clusters=i)
    km.fit_predict(X)
    wcss.append(km.inertia_)
```

Line-by-line Explanation

`wcss = []`

This creates an empty list to store WCSS values for the 3D dataset.

`for i in range(1,11):`

This loop tests the number of clusters from 1 to 10.

`km = KMeans(n_clusters=i)`

This creates a K-Means model with `i` clusters.

`km.fit_predict(X)`

This fits the model on the 3D data `X` and predicts cluster labels.

`wcss.append(km.inertia_)`

This stores the WCSS/inertia value for the current `K`.

Cell 21: Plotting the Elbow Graph for 3D Data

```
plt.plot(range(1,11),wcss)
```

Line-by-line Explanation

`plt.plot(range(1,11),wcss)`

This plots `K` values from 1 to 10 against WCSS values for the 3D data. Since the data was created with 4 centers, the elbow should appear around `K = 4`.

Cell 22: Applying Final K-Means to 3D Data

```
km = KMeans(n_clusters=4)
y_pred = km.fit_predict(X)
```

Line-by-line Explanation

```
km = KMeans(n_clusters=4)
```

This creates a K-Means model with 4 clusters.

```
y_pred = km.fit_predict(X)
```

This trains the K-Means model on the 3D data and stores the predicted cluster label for each point in y_pred.

Cell 23: Creating a DataFrame for 3D Visualization

```
df = pd.DataFrame()

df['col1'] = X[:,0]
df['col2'] = X[:,1]
df['col3'] = X[:,2]
df['label'] = y_pred
```

Line-by-line Explanation

```
df = pd.DataFrame()
```

This creates an empty DataFrame.

```
df['col1'] = X[:,0]
```

This creates a column named col1 and stores the first coordinate of each data point.

```
df['col2'] = X[:,1]
```

This creates a column named col2 and stores the second coordinate of each data point.

```
df['col3'] = X[:,2]
```

This creates a column named col3 and stores the third coordinate of each data point.

```
df['label'] = y_pred
```

This creates a column named label and stores the cluster number predicted by K-Means.

Cell 24: Final 3D Cluster Visualization

```
fig = px.scatter_3d(df, x='col1', y='col2', z='col3', color='label')
fig.show()
```

Line-by-line Explanation

```
fig = px.scatter_3d(df, x='col1', y='col2', z='col3', color='label')
```

This creates a 3D scatter plot using the DataFrame. col1 is x-axis, col2 is y-axis, col3 is z-axis, and color=label gives a different color to each cluster.

```
fig.show()
```

This displays the final interactive 3D cluster plot.

Final Summary of the Notebook

1. Import required libraries: NumPy, Pandas, Matplotlib, Scikit-learn, and Plotly.
2. Load student data from a CSV file.
3. Visualize student CGPA and IQ values using a scatter plot.
4. Use the Elbow Method to calculate WCSS for different values of K.
5. Choose K = 4 and train the final K-Means model.
6. Visualize the final student clusters using different colors.
7. Generate 3D synthetic data using make_blobs.
8. Apply the same K-Means process to the 3D data.
9. Visualize 3D clusters using Plotly.

Professional Improvement

In newer versions of Scikit-learn, it is better to set `random_state` and `n_init` explicitly. This makes the result more stable and reproducible.

```
km = KMeans(n_clusters=4, random_state=42, n_init=10)
y_means = km.fit_predict(X)
```

Why this improved version is better

- `random_state=42` makes the result reproducible, so the same code gives the same clusters each time.
- `n_init=10` runs K-Means multiple times with different initial centroids and keeps the best result.
- This is more professional for teaching, assignments, reports, and client-ready notebooks.